CEFET/RJ

Centro Federal de Educação Tecnológica Celso Suckow da Fonseca

Bacharelado em Ciência da Computação

GCC1734 – Inteligência Artificial

Modelos de Linguagem

Do Pré-Treinamento ao Alinhamento

Nota de Aula 08

Prof. Eduardo Bezerra

Escola de Informática e Computação

CEFET/RJ

11 de novembro de 2026

Sumário

1 Por que estudar o treinamento de modelos de linguagem? 2

2 Da Aproximação de Funções ao Modelo de Linguagem 3

3 Tokens e Representações Vetoriais 4

3.1 Tokenização 4

3.2 Embeddings 5

3.3 Informação de posição 7

4 Modelagem Autoregressiva da Linguagem 8

4.1 Probabilidade de uma sequência 9

4.2 Logits e softmax 10

4.3 Entropia cruzada 11

4.4 Exemplo resolvido: probabilidade e perda 12

4.5 Treinamento e geração não são a mesma coisa 13

5 A Arquitetura Transformer 15

5.1 A pergunta central da atenção 15

5.2 Máscara causal 16

5.3 Múltiplas cabeças e transformação por posição 16

5.4 Exemplo resolvido: combinação por atenção 17

6 Pré-Treinamento Autossupervisionado 17

6.1 O que o objetivo obriga o modelo a aprender? 17

6.2 O modelo não é uma base de dados textual 18

6.3 Escala, dados e capacidade 18

7 Ajuste Supervisionado para Seguir Instruções 19

7.1 Função de perda do SFT 19

7.2 O que muda depois do SFT? 19

7.3 Exemplo: pré-treinamento e SFT 20

8 Aprendizado por Reforço com Feedback Humano 20

8.1 De preferências humanas a uma função de recompensa 20

8.2 O processo de RLHF 21

8.3 RLHF como um problema de aprendizado por reforço 22

8.4 Exemplo resolvido: aprendizado de preferências sobre respostas 23

8.5 Limitações e cuidados . . . 26
8.6 RLHF e *reward shaping* . . . 26

9 Otimização da Política com PPO **27**
9.1 Otimização direta de uma política . . . 27
9.2 Política antiga e razão de probabilidades . . . 28
9.3 O objetivo com *clipping* . . . 28
9.4 Exemplo resolvido: efeito do *clipping* . . . 29
9.5 Duas noções diferentes de proximidade . . . 29
9.6 Fluxo conceitual do RLHF com PPO . . . 30

10 Otimização Direta de Preferências **31**
10.1 O que o DPO remove? . . . 31
10.2 RLHF e DPO não são sinônimos . . . 32

11 Comparação das Etapas de Treinamento **32**
11.1 Uma visão integrada . . . 33

12 Limitações e Fronteiras do Alinhamento **33**

13 Do Modelo de Linguagem ao Agente **34**

14 Resumo **35**

15 Atividades Intra-Classe **35**
15.1 Aquecimento . . . 35
15.2 Probabilidade de sequências . . . 36
15.3 Atenção e máscara causal . . . 36
15.4 Pré-treinamento, SFT e preferências . . . 37
15.5 PPO . . . 37
15.6 Projeto de dados de preferência . . . 37
15.7 Perguntas para discussão . . . 38

Objetivos

Ao final da leitura, você deve ser capaz de:

Fundamentos e representação

- Explicar por que modelos de linguagem representam texto como sequências de tokens.
- Descrever o papel de embeddings, posições e redes neurais na representação de uma sequência.
- Distinguir parâmetros aprendidos, arquitetura e dados de treinamento.

Modelagem autoregressiva ⋆

- Interpretar um modelo de linguagem como uma distribuição de probabilidade sobre o próximo token.
- Decompor a probabilidade de uma sequência pelo produto das probabilidades condicionais de seus tokens.
- Explicar a relação entre logits, softmax, entropia cruzada e predição do próximo token.
- Distinguir treinamento, em que o próximo token correto é conhecido, de geração, em que o próprio modelo escolhe a continuação.

Transformer e pré-treinamento ⋆

- Explicar conceitualmente como a autoatenção permite combinar informações de diferentes posições da sequência.
- Descrever os papéis de consultas, chaves e valores, sem derivar a arquitetura completa.
- Explicar por que a máscara causal impede o modelo de observar tokens futuros.
- Caracterizar o pré-treinamento como aprendizado autossupervisionado em grande escala.

Pós-treinamento e alinhamento ⋆

- Distinguir pré-treinamento, ajuste supervisionado e otimização por preferências.
- Explicar como comparações humanas podem ser convertidas em um modelo de recompensa.
- Formular a geração de texto como um problema de decisão sequencial.
- Explicar o papel do PPO, do *clipping* e da penalização por divergência KL no RLHF.

- Contrastar RLHF e Otimização Direta de Preferências (DPO).

Limites e conexão com agentes

- Explicar por que maximizar a recompensa prevista não garante respostas verdadeiras ou seguras.
- Distinguir um modelo de linguagem de um agente que usa o modelo para decidir, planejar e operar ferramentas.

Roteiro de leitura. A nota começa com a representação numérica do texto e o papel de redes neurais como aproximadores de funções. Em seguida, apresenta a modelagem autoregressiva, a arquitetura Transformer e o pré-treinamento por predição do próximo token. Depois, discute o ajuste supervisionado e o treinamento a partir de preferências humanas. A parte final abre conceitualmente o mecanismo de otimização usado no RLHF, introduz o PPO, apresenta o DPO como alternativa e delimita a passagem de modelos de linguagem para agentes baseados em LLMs. Os objetivos marcados com $\star$ constituem o núcleo da nota.

1 Por que estudar o treinamento de modelos de linguagem?

Modelos de linguagem de grande porte são componentes centrais de sistemas contemporâneos de IA. Eles produzem texto, código e representações úteis para diversas tarefas. Também podem participar de sistemas que consultam bases de conhecimento, utilizam ferramentas, mantêm memória e executam planos. Entretanto, essas capacidades não surgem de uma única etapa de treinamento. É útil separar três perguntas, que correspondem a diferentes etapas do treinamento de um modelo de linguagem:

1. **Como o modelo aprende regularidades da linguagem e conhecimento presente nos dados?** Por meio do **pré-treinamento**, geralmente baseado na predição do próximo token. Ao processar grandes quantidades de texto, o modelo aprende, por exemplo, que a expressão "aprendizado por reforço" costuma aparecer associada a conceitos como agentes, ações e recompensas. Essas regularidades são aprendidas sem que alguém precise ensinar explicitamente cada conceito ou fornecer perguntas e respostas.
2. **Como o modelo aprende a responder a instruções?** Por meio do **ajuste supervisionado**, utilizando exemplos de pares instrução–resposta. Considere a instrução "Explique aprendizado por reforço em duas frases". O pré-treinamento pode ter fornecido conhecimento sobre o assunto, mas não garante que o modelo interprete o texto como uma solicitação. Ao observar exemplos de instruções acompanhadas de respostas adequadas, o modelo aprende a produzir respostas que atendam ao pedido, inclusive quanto ao formato solicitado.
3. **Como o comportamento é aproximado das preferências humanas?** Por meio do **alinhamento por preferências**, utilizando métodos como RLHF ou DPO.

Para a mesma instrução, o modelo pode produzir duas respostas que respeitam o limite de duas frases, mas diferem em clareza e correção. Avaliadores humanos podem indicar qual delas preferem. Essas comparações fornecem um sinal adicional de treinamento, utilizado para favorecer respostas consideradas melhores segundo os critérios adotados.

Essas etapas possuem objetivos diferentes. O pré-treinamento favorece a continuação provável de sequências. O ajuste supervisionado favorece a imitação de respostas demonstradas. A otimização por preferências favorece respostas avaliadas como melhores segundo algum sinal humano ou aprendido. Confundir essas etapas dificulta entender tanto as capacidades quanto as limitações dos LLMs.

Também é necessário distinguir **modelo** e **agente**. Um LLM recebe um contexto e produz uma distribuição sobre continuações possíveis. Um agente incorpora esse modelo em um ciclo mais amplo: observa uma situação, decide o que fazer, utiliza ferramentas, recebe novos resultados e continua o processo. Esta nota concentra-se na construção e no pós-treinamento do modelo. A arquitetura dos agentes será estudada posteriormente.

Pergunta 1

Um modelo foi treinado em grande quantidade de textos, mas frequentemente ignora instruções como “responda em duas frases”. Que etapa de treinamento parece insuficiente: pré-treinamento, ajuste supervisionado ou otimização por preferências? Justifique sem assumir que exista uma única resposta possível.

2 Da Aproximação de Funções ao Modelo de Linguagem

Na Nota de Aula 07, vimos que uma rede neural pode ser usada como uma função parametrizada:

$$f_\theta(x),$$

onde x representa a entrada e θ reúne os parâmetros ajustados durante o treinamento. Em um modelo de linguagem, a entrada não é apenas um vetor fixo de características: é uma sequência de tokens transformada em embeddings e representações intermediárias.

O modelo precisa compartilhar os mesmos parâmetros entre muitas posições e sequências. Esse compartilhamento evita manter uma tabela independente para cada contexto possível e permite generalizar entre contextos relacionados. As representações intermediárias não são fornecidas como rótulos: elas são aprendidas porque ajudam a produzir previsões melhores para a tarefa definida.

O fluxo geral pode ser resumido assim:

1. o texto é convertido em *tokens* e, depois, em *embeddings*;
2. a rede transforma a sequência em *representações contextuais*;

3. uma camada de saída produz logits para os tokens do vocabulário;
4. a softmax converte os logits em uma distribuição de probabilidade;
5. a perda de predição do próximo token orienta o ajuste dos parâmetros.

O mecanismo de treinamento continua sendo o mesmo estudado anteriormente: uma função de perda define o erro, a retropropagação calcula os gradientes e um otimizador atualiza os parâmetros. A diferença está na tarefa e na escala: em vez de uma saída para um vetor de características, o modelo produz distribuições sobre o vocabulário ao longo de uma sequência. As próximas seções detalham como tokens, embeddings, posição, atenção e modelagem autoregressiva implementam esse fluxo.

Pergunta 2

Compare o fluxo de um classificador MLP estudado na Nota 07 com o de um modelo de linguagem. O que permanece igual em ambos os casos quanto a parâmetros, logits, perda e retropropagação? O que muda quando a entrada passa a ser uma sequência de tokens e a saída passa a ser uma distribuição sobre o vocabulário em cada posição?

3 Tokens e Representações Vetoriais

Modelos de linguagem não recebem palavras diretamente. Antes do processamento, o texto é convertido em uma sequência de unidades discretas denominadas *tokens*. Cada token é associado a um identificador inteiro, que permite ao modelo representá-lo e processá-lo numericamente.

Para isso, cada modelo utiliza um *vocabulário*: um conjunto finito de tokens reconhecidos pelo seu tokenizador. Cada elemento desse vocabulário possui um identificador inteiro único, denominado *token ID*. Por exemplo, em um vocabulário hipotético, o token `aprend` poderia receber o identificador 125, enquanto `izado` receberia 387. Esses números funcionam apenas como índices; seus valores não representam o significado dos tokens.

3.1 Tokenização

A *tokenização* é o processo de decompor um texto em tokens pertencentes ao vocabulário do modelo. Um token pode corresponder a uma palavra inteira, parte de uma palavra, sinal de pontuação, espaço ou sequência frequente de caracteres.

Considere a frase:

`aprendizado por reforço`

Um tokenizador hipotético poderia decompô-la na seguinte sequência:

[`aprend`, `izado`, `por`, `refor`, `ço`].

Suponha que esses cinco tokens estejam associados aos seguintes identificadores no vocabulário:

Token	Identificador
`aprend`	125
`izado`	387
` por`	42
` refor`	918
`ço`	203

Nesse caso, a frase seria representada inicialmente pela sequência de inteiros:

$$[125,\ 387,\ 42,\ 918,\ 203].$$

Observe que os espaços anteriores a `por` e `refor` fazem parte dos respectivos tokens. Isso permite que o tokenizador preserve informações necessárias para reconstruir o texto original.

A decomposição real depende do tokenizador e de seu vocabulário. Métodos baseados em subpalavras procuram equilibrar duas necessidades. Um vocabulário muito pequeno tende a produzir sequências longas, pois exige dividir o texto em unidades menores. Um vocabulário muito grande tende a consumir mais parâmetros, sem necessariamente conseguir representar cada palavra, flexão ou nome como um único token.

A tokenização afeta o custo computacional e a forma como o modelo representa padrões. Duas palavras visualmente semelhantes podem ser segmentadas de maneiras diferentes. Textos em idiomas menos representados nos dados de construção do tokenizador podem exigir mais tokens para expressar o mesmo conteúdo.

3.2 Embeddings

Na subseção anterior, vimos que cada token é associado a um identificador inteiro. Entretanto, esses identificadores funcionam apenas como índices no vocabulário. Por exemplo, se os tokens `gato`, `cachorro` e `computador` possuem identificadores 125, 126 e 127, respectivamente, não podemos concluir que `gato` e `cachorro` sejam mais semelhantes entre si apenas porque seus identificadores estão numericamente próximos. A atribuição desses números não codifica o significado dos tokens.

Essa limitação é importante porque os tokens não são utilizados de maneira independente na linguagem. Algumas palavras aparecem em contextos semelhantes ou desempenham funções relacionadas. Por exemplo, `gato` e `cachorro` podem aparecer em frases como “O gato dormiu no sofá” e “O cachorro dormiu no sofá”. Poderíamos até construir a frase “O computador dormiu no sofá”, que é gramaticalmente correta, mas semanticamente pouco plausível. Isso mostra que ocupar a mesma posição em uma estrutura gramatical não significa ter o mesmo significado. Enquanto `gato` e `cachorro` compartilham diversos contextos de uso, `computador` costuma aparecer associado a programas, arquivos e processamento de dados.

Seria útil, portanto, representar os tokens de maneira que essas relações pudessem ser aproveitadas pelo modelo. Em particular, gostaríamos que tokens que compartilham regularidades de significado ou de uso pudessem adquirir representações numericamente próximas. Quando essa proximidade reflete semelhanças de significado, falamos em *proximidade semântica*. Entretanto, as relações representadas não precisam ser exclusivamente semânticas: também podem envolver propriedades gramaticais e outras regularidades presentes nos dados.

Para construir essas representações, o modelo utiliza *embeddings*. Um embedding é um vetor de números reais associado a um token. Em vez de utilizar diretamente o identificador 125 para representar `gato`, por exemplo, o modelo poderia associá-lo ao vetor hipotético

$$\mathbf{e}_{125} = (0{,}8,\ 0{,}2,\ 0{,}7).$$

O token `cachorro` poderia receber um vetor semelhante, como $(0{,}7,\ 0{,}3,\ 0{,}8)$, enquanto `computador` poderia receber $(0{,}1,\ 0{,}9,\ 0{,}2)$. Nesse exemplo ilustrativo, os dois primeiros vetores estão mais próximos entre si do que do terceiro, considerando a distância euclidiana. Diferentemente dos identificadores inteiros, esses vetores permitem expressar numericamente relações entre tokens.

Os embeddings são organizados em uma matriz:

$$E \in \mathbb{R}^{|V| \times d},$$

onde $|V|$ representa a quantidade de tokens no vocabulário e d representa a quantidade de componentes de cada vetor. Cada linha E_i contém o embedding associado ao token de identificador i. Assim, quando o tokenizador produz uma sequência de identificadores, o modelo utiliza esses índices para recuperar os vetores correspondentes na matriz.

Uma característica fundamental é que *os embeddings são aprendidos durante o treinamento*. Seus valores não precisam ser definidos manualmente. Inicialmente, os vetores podem conter valores aleatórios. À medida que o modelo ajusta seus parâmetros para melhorar suas previsões, esses vetores também são modificados. Tokens utilizados em contextos relacionados podem adquirir representações que permitem ao restante da rede explorar essas relações.

Por fim, é importante não interpretar cada componente de um embedding como uma característica previamente definida. Não existe, por exemplo, uma dimensão necessariamente dedicada a indicar se um token representa um animal. As informações são distribuídas entre diversas componentes, e suas relações emergem do processo de treinamento. Além disso, a proximidade entre embeddings não garante equivalência de significado: ela pode refletir diferentes tipos de regularidade aprendidos a partir dos dados. Por exemplo, os tokens `gato` e `cachorro` podem adquirir representações próximas por compartilharem contextos de uso, mas isso também pode ocorrer com `gato` e `mesa`, que possuem significados distintos, mas pertencem à mesma classe gramatical e podem ocupar posições semelhantes em uma frase.

Os embeddings constituem, portanto, a primeira representação vetorial dos tokens. Nas etapas seguintes do processamento, essas representações serão transformadas levando em consideração os demais tokens da sequência. Assim, embora um token possua inicialmente

um vetor associado ao seu identificador, sua representação posterior poderá depender do contexto em que aparece.

3.3 Informação de posição

Na subseção anterior, vimos que cada token do vocabulário possui um embedding, isto é, um vetor aprendido durante o treinamento. Entretanto, esse vetor é inicialmente o mesmo independentemente da posição ocupada pelo token no texto. Isso significa que, se considerarmos apenas os embeddings, ainda não teremos informações suficientes sobre a ordem dos tokens na sequência.

Considere as frases:

o aluno corrigiu o modelo e *o modelo corrigiu o aluno*

As duas frases contêm exatamente os mesmos tokens, mas expressam situações diferentes. Na primeira, o aluno realiza a ação de corrigir; na segunda, quem realiza essa ação é o modelo. A diferença de significado decorre da posição que cada palavra ocupa na frase.

Ao converter essas frases em embeddings, o token `aluno` será inicialmente representado pelo mesmo vetor nas duas sequências, assim como os demais tokens. O que muda é a ordem em que esses vetores aparecem. Portanto, para interpretar corretamente as frases, o modelo precisa considerar não apenas quais tokens estão presentes, mas também como eles estão organizados.

Essa necessidade motiva a incorporação de *informação posicional*. Seu objetivo é permitir que o processamento leve em conta a posição dos tokens na sequência. Essa informação pode indicar posições absolutas, como primeiro, segundo e terceiro token, ou relações entre posições, como a distância e a ordem relativa entre dois tokens.

Uma forma de incorporar essa informação consiste em associar a cada posição um vetor posicional e combiná-lo com o embedding do token correspondente. Assim, a representação utilizada pelo modelo passa a carregar informações tanto sobre o token quanto sobre sua posição. Por exemplo, o token `aluno`, quando aparece no início da frase, recebe uma representação diferente daquela utilizada quando aparece no final.

Essa não é, entretanto, a única maneira de representar posições. Algumas arquiteturas incorporam a informação posicional diretamente nas operações de atenção, como ocorre com o método *Rotary Position Embedding* (RoPE). Embora os mecanismos sejam diferentes, todos procuram permitir que o modelo utilize a organização dos tokens ao interpretar uma sequência.

A Figura 1 resume o percurso apresentado até aqui: o texto é dividido em tokens, cada token é convertido em um identificador inteiro, e esses identificadores permitem recuperar os embeddings correspondentes. A informação posicional é então incorporada ao processamento, permitindo que o modelo considere tanto o conteúdo quanto a ordem dos tokens.

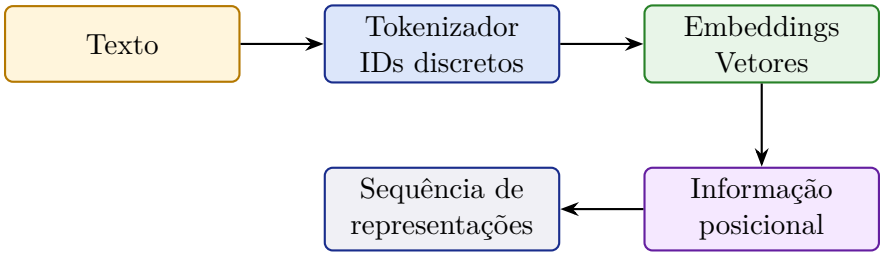


Figura 1: Representação numérica de uma sequência de texto. O tokenizador produz identificadores que permitem recuperar os embeddings aprendidos. A incorporação da informação posicional é mostrada como uma etapa separada para fins didáticos; em arquiteturas como aquelas que utilizam RoPE, essa informação é introduzida diretamente nas operações de atenção.

Pergunta 3

Considere as frases "O professor avaliou o aluno" e "O aluno avaliou o professor". Se ambas contêm os mesmos tokens e cada token possui um embedding inicialmente fixo, por que o modelo precisa de informação posicional para distinguir seus significados? O que poderia acontecer se o processamento não levasse em conta a ordem dos tokens?

4 Modelagem Autoregressiva da Linguagem

Nas seções anteriores, vimos como um texto é convertido em uma sequência de tokens e como esses tokens recebem representações vetoriais. Resta compreender o que o modelo faz com essas representações e, principalmente, como aprende a produzir texto.

Uma das abordagens mais utilizadas em modelos de linguagem de grande porte é a *modelagem autoregressiva*. A ideia central é simples: dado um trecho de texto, o modelo procura prever qual token provavelmente aparecerá em seguida.

Considere, por exemplo, o trecho:

aprendizado por

Dependendo das regularidades aprendidas durante o treinamento, o modelo poderá atribuir probabilidades diferentes a possíveis continuações desse trecho, tais como `reforço`, `exemplo` ou `máquina`. Em vez de produzir diretamente uma única resposta, ele calcula uma *distribuição de probabilidade* sobre os tokens de seu vocabulário. Essa distribuição indica quão provável o modelo considera cada possível próximo token, dado o texto recebido.

O trecho de texto utilizado para realizar essa previsão é denominado contexto. Na modelagem autoregressiva, o modelo é treinado para estimar a probabilidade do próximo token, dados os tokens anteriores. Representando os tokens do contexto por $x_1, \ldots, x_t$, essa probabilidade é expressa por:

$$P_\theta(x_{t+1} \mid x_1, \ldots, x_t),$$

onde x_{t+1} representa o próximo token e θ reúne os parâmetros aprendidos durante o treinamento. A expressão indica a probabilidade atribuída a um possível próximo token, considerando os tokens anteriores.

Para produzir uma resposta, o modelo seleciona um token a partir dessa distribuição e o acrescenta ao contexto. Em seguida, calcula uma nova distribuição, agora considerando também o token recém-adicionado. Esse procedimento se repete, construindo o texto progressivamente, até que seja produzido um token de término ou que algum limite seja alcançado.

O termo *autoregressivo* está relacionado justamente a esse mecanismo: cada nova previsão depende dos tokens anteriores, incluindo aqueles que o próprio modelo já produziu.

4.1 Probabilidade de uma sequência

Até aqui, consideramos a probabilidade de um único token. Entretanto, um texto é formado por uma sequência de tokens. Como podemos determinar a probabilidade que o modelo atribui a uma sequência completa?

Considere novamente a expressão:

aprendizado por reforço

Para simplificar, suponha que ela seja representada por três tokens: `aprendizado`, `por` e `reforço`. A probabilidade de produzir essa sequência pode ser calculada multiplicando três probabilidades: a de produzir `aprendizado` inicialmente, a de produzir `por` depois de `aprendizado` e a de produzir `reforço` depois de `aprendizado por`.

Em termos matemáticos:

$$\begin{aligned} &P_\theta(\texttt{aprendizado}, \texttt{por}, \texttt{reforço}) \\ &= P_\theta(\texttt{aprendizado}) \\ &\quad \times P_\theta(\texttt{por} \mid \texttt{aprendizado}) \\ &\quad \times P_\theta(\texttt{reforço} \mid \texttt{aprendizado}, \texttt{por}). \end{aligned}$$

Essa decomposição decorre da *regra da cadeia das probabilidades*, também conhecida como aplicação sucessiva da regra do produto. Para uma sequência genérica de T tokens, temos:[1]

$$P_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} P_\theta(x_t \mid x_1, \ldots, x_{t-1}).$$

Para $t = 1$, o contexto anterior é vazio; em implementações práticas, pode-se utilizar um token especial de início de sequência ou outro contexto inicial.

Essa expressão mostra que não precisamos calcular diretamente a probabilidade de cada sequência completa possível. Podemos decompor o problema em uma sucessão de previsões, cada uma condicionada aos tokens anteriores.

[1] Essa regra da cadeia é uma identidade de probabilidade. Ela não deve ser confundida com a regra da cadeia do cálculo, utilizada para obter gradientes na retropropagação.

Essa propriedade também fornece uma maneira de treinar o modelo. Considere a sequência:

aprendizado por reforço usa recompensas

Durante o treinamento, o texto já é conhecido. Assim, o modelo pode receber o contexto "aprendizado por" e tentar prever o próximo token observado, "reforço". Da mesma forma, ao receber "aprendizado por reforço usa", deve atribuir uma probabilidade elevada ao token "recompensas".

Cada posição da sequência fornece, portanto, um exemplo de treinamento: os tokens anteriores constituem a entrada, e o próximo token observado constitui o alvo. O modelo aprende ajustando seus parâmetros para aumentar, em média, as probabilidades atribuídas aos tokens efetivamente observados.

Como exemplo, considere novamente a sequência "aprendizado por reforço usa recompensas". Para simplificar, suponha que cada palavra corresponda a um único token. A tabela abaixo mostra os exemplos de treinamento que podem ser construídos a partir dessa sequência.

Posição	**Contexto (entrada)**	**Próximo token (alvo)**
1	`aprendizado`	`por`
2	`aprendizado por`	`reforço`
3	`aprendizado por reforço`	`usa`
4	`aprendizado por reforço usa`	`recompensas`

Observe que o contexto aumenta progressivamente, enquanto o alvo corresponde sempre ao token imediatamente seguinte. Durante o treinamento, o modelo procura atribuir probabilidade elevada a cada um desses alvos, considerando o respectivo contexto. Assim, uma sequência de cinco tokens fornece quatro exemplos de predição, sem necessidade de rotulagem manual.

4.2 Logits e softmax

Vimos que o modelo precisa produzir uma distribuição de probabilidade sobre os possíveis próximos tokens. Mas como uma rede neural produz essa distribuição?

Após processar as representações do contexto, a rede produz um conjunto de valores numéricos, um para cada token do vocabulário. Esses valores são denominados *logits*.

Se o vocabulário possui $|V|$ tokens, a saída pode ser representada pelo vetor:

$$\mathbf{z} = (z_1, \ldots, z_{|V|}).$$

Cada componente z_i corresponde ao logit associado ao token de identificador i. Valores maiores indicam que o modelo favorece aquele token em relação a outros. Entretanto, os logits ainda não são probabilidades: podem ser negativos, não estão restritos ao intervalo entre 0 e 1 e não precisam somar 1.

Para transformá-los em uma distribuição de probabilidade, utiliza-se a função *softmax*:

$$P_\theta(x_{t+1} = i \mid x_{1:t}) = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}.$$

A expressão calcula a probabilidade atribuída ao token i. O numerador considera o logit desse token, enquanto o denominador considera os logits de todos os tokens do vocabulário. Como resultado, cada probabilidade fica entre 0 e 1, e a soma das probabilidades é igual a 1.

Por exemplo, suponha que o contexto seja "aprendizado por" e que, em um vocabulário hipotético de apenas três tokens, a rede produza os seguintes logits:

Token	**Logit**
`reforço`	2
`máquina`	1
`exemplo`	0

Aplicando a função softmax, obtemos aproximadamente:

$$P(\texttt{reforço}) = 0{,}665,$$
$$P(\texttt{máquina}) = 0{,}245,$$
$$P(\texttt{exemplo}) = 0{,}090.$$

Observe que `reforço` recebeu a maior probabilidade, mas as outras possibilidades não foram descartadas. O modelo expressa, assim, diferentes graus de plausibilidade para as continuações do contexto.

4.3 Entropia cruzada

Agora sabemos como o modelo produz probabilidades. Falta compreender como essas probabilidades são utilizadas para orientar o aprendizado.

Durante o treinamento, conhecemos o token que efetivamente aparece após cada contexto. Podemos, portanto, comparar a probabilidade atribuída a esse token com o resultado observado.

Considere novamente o contexto "aprendizado por", cujo próximo token no texto de treinamento é "reforço". Se o modelo atribuir probabilidade elevada a `reforço`, sua previsão será considerada adequada. Se atribuir probabilidade muito baixa, deverá receber uma penalização maior.

Essa penalização é calculada por uma *função de perda*. Na modelagem autoregressiva, utiliza-se normalmente a *entropia cruzada*, que, para um único token observado, pode ser escrita como:

$$L_t(\theta) = -\log P_\theta(y \mid x_{1:t}),$$

onde y é o token efetivamente observado após o contexto $x_{1:t}$.

O logaritmo transforma probabilidades em valores adequados para a função de perda. Como probabilidades estão entre 0 e 1, seus logaritmos são não positivos. O sinal negativo faz com que a perda seja não negativa.

O comportamento dessa função é especialmente importante:

- Se o modelo atribui probabilidade próxima de 1 ao token observado, a perda é pequena.
- Se atribui probabilidade baixa ao token observado, a perda é elevada.
- Quanto menor a probabilidade atribuída ao token correto, maior será a penalização.

Por exemplo, se o modelo atribuir probabilidade 0,8 ao token observado, a perda será aproximadamente 0,223. Se atribuir apenas 0,1, a perda aumentará para aproximadamente 2,303.

Durante o treinamento, essas perdas são agregadas ao longo das posições e dos exemplos de um *minibatch*. O gradiente dessa perda é então utilizado para ajustar os parâmetros da rede, conforme o procedimento de gradiente descendente e retropropagação apresentado anteriormente.

Portanto, o aprendizado autoregressivo pode ser resumido como um processo repetido: o modelo recebe um contexto, prevê probabilidades para o próximo token, compara sua previsão com o token observado e ajusta seus parâmetros para reduzir a perda. Ao repetir esse processo sobre grandes quantidades de texto, aprende regularidades que o ajudam a prever continuações em contextos cada vez mais variados.

4.4 Exemplo resolvido: probabilidade e perda

Vimos que, durante o treinamento, o modelo recebe um contexto, produz uma distribuição de probabilidade sobre os possíveis próximos tokens e calcula uma perda com base no token efetivamente observado. Vamos ilustrar esse processo com um exemplo numérico.

Considere que o texto de treinamento contenha a expressão “aprendizado por reforço”. Ao receber o contexto “aprendizado por”, o modelo deve prever o próximo token, que, nesse caso, é `reforço`.

Para simplificar, suponha que o vocabulário contenha apenas quatro tokens e que o modelo produza a seguinte distribuição de probabilidade:

Token	**Probabilidade prevista**	**É o alvo observado?**
`reforço`	0,70	Sim
`exemplo`	0,15	Não
`modelo`	0,10	Não
`fim`	0,05	Não

Observe que as probabilidades somam 1 e que o modelo atribuiu a maior probabilidade ao token `reforço`. Entretanto, durante o treinamento, não basta verificar qual token recebeu

a maior probabilidade. Queremos também avaliar *quanta confiança o modelo atribuiu ao token efetivamente observado.*

Como o alvo é `reforço` e sua probabilidade prevista é 0,70, a perda de entropia cruzada será:

$$L = -\log(0{,}70) \approx 0{,}357.$$

Suponha agora que, após receber o mesmo contexto, o modelo produzisse uma distribuição diferente, atribuindo probabilidade de apenas 0,10 ao token `reforço`. Nesse caso, a perda seria:

$$L = -\log(0{,}10) \approx 2{,}303.$$

A segunda previsão recebe uma penalização muito maior porque o modelo atribuiu baixa probabilidade ao token que efetivamente apareceu no texto. Por outro lado, se tivesse atribuído probabilidade 0,95 a `reforço`, a perda seria de apenas:

$$L = -\log(0{,}95) \approx 0{,}051.$$

Portanto, quanto maior a probabilidade atribuída ao token observado, menor será a perda. Esse mecanismo fornece um sinal numérico que permite avaliar as previsões do modelo e orientar seu aprendizado.

Durante o treinamento, a perda calculada para esse exemplo é combinada com as perdas de outras posições e sequências. Por meio da retropropagação, são calculados os gradientes que indicam como os parâmetros da rede influenciam essas perdas. O algoritmo de otimização utiliza esses gradientes para atualizar os parâmetros, buscando reduzir a perda média.

Assim, o modelo não recebe uma regra explícita determinando que a palavra `reforço` deve aparecer depois de "aprendizado por". Ele aprende essa e muitas outras regularidades ao ajustar repetidamente seus parâmetros para aumentar as probabilidades dos tokens observados nos dados de treinamento.

4.5 Treinamento e geração não são a mesma coisa

Até aqui, vimos como um modelo autoregressivo aprende a prever o próximo token a partir de um contexto. Entretanto, existe uma diferença importante entre a maneira como essas previsões são realizadas durante o treinamento e durante a geração de texto.

Durante o treinamento, o modelo recebe sequências extraídas dos dados e procura prever cada token a partir dos tokens anteriores. O contexto utilizado em cada previsão vem, portanto, do próprio texto de treinamento. Esse procedimento é conhecido como *teacher forcing*: independentemente da previsão realizada pelo modelo, o contexto da posição seguinte continua sendo formado pelos tokens efetivamente observados.

Durante a geração, por outro lado, o modelo não conhece antecipadamente os tokens que formarão a resposta. Ele precisa selecionar um token, acrescentá-lo ao contexto e utilizar a sequência resultante para realizar a próxima previsão. Assim, cada escolha influencia as probabilidades das escolhas seguintes.

Considere, por exemplo, que o texto de treinamento contenha a sequência "aprendizado por reforço usa recompensas". A tabela abaixo ilustra a diferença entre os dois processos, supondo que cada palavra corresponda a um token.

Etapa	**Treinamento (*teacher forcing*)**	**Geração autoregressiva**
1	Contexto: `aprendizado por`. Alvo: `reforço`.	Contexto: `aprendizado por`. Token escolhido: `exemplo`.
2	Contexto: `aprendizado por reforço`. Alvo: `usa`.	Contexto: `aprendizado por exemplo`. O modelo escolhe o próximo token.
3	Contexto: `aprendizado por reforço usa`. Alvo: `recompensas`.	O contexto já contém os tokens produzidos nas etapas anteriores.

Observe que, durante o treinamento, mesmo que o modelo atribua maior probabilidade a `exemplo` do que a `reforço`, a previsão seguinte continua utilizando o contexto correto, "aprendizado por reforço". Na geração, se o modelo selecionar `exemplo`, será esse token que passará a integrar o contexto.

Essa diferença pode produzir um fenômeno conhecido como *exposure bias*. Durante o treinamento, o modelo aprende a realizar previsões condicionadas aos contextos presentes nos dados. Na geração, entretanto, pode encontrar contextos criados por suas próprias escolhas, inclusive sequências pouco representadas durante o treinamento. Uma escolha inadequada pode, assim, comprometer as previsões seguintes. Isso não significa que qualquer continuação diferente do texto de treinamento seja um erro: frequentemente existem várias continuações válidas para um mesmo contexto.

Como o modelo escolhe o próximo token?

Vimos que, para cada contexto, o modelo produz uma distribuição de probabilidade sobre os tokens do vocabulário. Durante a geração, é necessário utilizar essa distribuição para escolher o próximo token. Algumas estratégias comuns são:

- **Escolha gulosa**: seleciona sempre o token com a maior probabilidade prevista. É uma estratégia determinística, mas a melhor escolha imediata não garante a sequência completa mais provável.
- **Amostragem**: sorteia o próximo token de acordo com a distribuição prevista. Tokens mais prováveis têm maior chance de serem escolhidos, mas tokens menos prováveis também podem ser selecionados.
- **Temperatura**: modifica a concentração da distribuição antes da amostragem. Temperaturas menores que 1 favorecem os tokens mais prováveis, enquanto temperaturas maiores que 1 tornam a distribuição mais uniforme, aumentando a diversidade das escolhas.
- **Top-k**: restringe a amostragem aos k tokens com maiores probabilidades, descartando temporariamente os demais.
- **Top-p**: restringe a amostragem ao menor conjunto de tokens mais prováveis cuja probabilidade acumulada atinge ou ultrapassa um limiar p.

Por exemplo, suponha que o modelo atribua probabilidades 0,50, 0,30, 0,15 e 0,05 a quatro possíveis tokens. A escolha gulosa selecionaria sempre o primeiro. Com top-k e $k = 2$, apenas os dois primeiros participariam da amostragem. Com top-p e $p = 0{,}80$, esses mesmos dois tokens seriam mantidos, pois suas probabilidades acumuladas somam 0,80. Em ambos os casos, as probabilidades dos tokens restantes são renormalizadas para que somem 1.

Essas estratégias podem ser combinadas e atuam apenas durante a geração: elas não modificam os parâmetros aprendidos pelo modelo. Sua escolha influencia características como diversidade, previsibilidade e coerência do texto produzido. Nenhuma delas, entretanto, garante que a resposta seja correta ou que a sequência escolhida seja globalmente a mais provável.

Pergunta 4

Um estudante afirma: "Se o modelo sempre escolher o token de maior probabilidade, produzirá necessariamente a resposta globalmente mais provável". Por que essa conclusão não decorre da escolha gulosa? Considere que cada token escolhido modifica as probabilidades das etapas seguintes.

5 A Arquitetura Transformer

O Transformer foi introduzido como uma arquitetura baseada em mecanismos de atenção, sem depender de recorrência para combinar posições da sequência (Vaswani et al., 2017). Em modelos autoregressivos do tipo *decoder-only*, blocos Transformer processam o contexto e produzem representações usadas para prever o próximo token.

5.1 A pergunta central da atenção

Para atualizar a representação de um token, o modelo precisa decidir quais outras posições do contexto são relevantes. Na frase

A aluna entregou o trabalho porque ela terminou a revisão,

a interpretação de "ela" depende de informações anteriores. A atenção permite que cada posição combine, com pesos diferentes, as representações das demais posições permitidas.

Para cada token, são produzidos três vetores:

- **consulta** (*query*, q): representa o tipo de informação buscada pela posição atual;
- **chave** (*key*, k): representa a informação usada para determinar a relevância de cada posição;
- **valor** (*value*, v): contém a informação que poderá ser incorporada à nova representação.

Consultas, chaves e valores são projeções aprendidas das representações de entrada:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

Os escores de compatibilidade são calculados por produtos internos entre consultas e chaves. Na atenção por produto escalar escalonado:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Aqui, M é a máscara de atenção. Em uma atenção causal, $M_{ij} = -\infty$ quando $j > i$ e $M_{ij} = 0$ nos demais casos. O softmax transforma os escores permitidos em pesos que somam 1. A saída de cada posição é uma combinação ponderada dos vetores de valor.

5.2 Máscara causal

Durante a predição, a saída associada à posição t é usada para prever o token seguinte, x_{t+1}. O modelo não pode consultar posições posteriores ao prefixo disponível. Se tivesse acesso ao futuro, poderia simplesmente observar a resposta usada como alvo. Uma **máscara causal** impede que a posição t atribua atenção às posições $t + 1, t + 2, \ldots$.

Essa restrição mantém a coerência entre treinamento e geração. O modelo aprende a prever cada token usando apenas o prefixo que estaria disponível no momento em que esse token precisasse ser gerado.

5.3 Múltiplas cabeças e transformação por posição

O mecanismo de atenção é executado em várias cabeças. Cada cabeça possui suas próprias projeções e pode aprender diferentes padrões de relacionamento. As saídas são combinadas e transformadas novamente.

Depois da atenção, cada posição passa por uma rede neural feed-forward aplicada separadamente à sua representação. Embora seja aplicada independentemente a cada posição, essa sub-rede não é necessariamente pequena e, em muitas configurações, concentra uma parcela substancial dos parâmetros do bloco. Conexões residuais e normalização ajudam a preservar e estabilizar o fluxo de informação. Um bloco Transformer contém, portanto, dois mecanismos principais:

1. comunicação entre posições por autoatenção;
2. transformação não linear da representação de cada posição.

O empilhamento de muitos blocos permite que informações sejam combinadas repetidamente.

5.4 Exemplo resolvido: combinação por atenção

Considere uma única cabeça de atenção. Para simplificar, suponha que a posição atual atribua pesos 0,60, 0,30 e 0,10 a três posições anteriores, cujos vetores de valor são:

$$v_1 = (1, 0), \qquad v_2 = (0, 2), \qquad v_3 = (1, 1).$$

A representação produzida pela atenção será:

$$\begin{aligned} h &= 0{,}60v_1 + 0{,}30v_2 + 0{,}10v_3 \\ &= 0{,}60(1, 0) + 0{,}30(0, 2) + 0{,}10(1, 1) \\ &= (0{,}60, 0) + (0, 0{,}60) + (0{,}10, 0{,}10) \\ &= (0{,}70, 0{,}70). \end{aligned}$$

O resultado não copia uma única posição. Ele combina informação das três, dando maior influência à primeira. Durante o treinamento, os parâmetros que geram consultas, chaves e valores são ajustados para que essas combinações ajudem a prever os próximos tokens.

Pergunta 5

Atenção elevada entre dois tokens prova que um deles causou a decisão final do modelo? Explique a diferença entre observar um peso interno e estabelecer uma explicação causal para a saída.

6 Pré-Treinamento Autossupervisionado

No pré-treinamento, o modelo processa grandes coleções de sequências e aprende a prever o próximo token: o próprio texto, deslocado uma posição, fornece tanto as entradas quanto os alvos. Não é necessário que uma pessoa rotule manualmente cada exemplo. Por isso, o procedimento é denominado **autossupervisionado**.

Em um modelo autoregressivo, uma sequência com T tokens, sem um token inicial especial, oferece $T - 1$ problemas de predição entre tokens consecutivos. Para escrever a fórmula com T alvos, podemos incluir um token de início, BOS, como x_0; nesse caso, a posição t prevê x_t a partir de $x_{<t}$, e o treinamento minimiza:

$$L_{\mathrm{PT}}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

6.1 O que o objetivo obriga o modelo a aprender?

Para prever continuações em muitos domínios, o modelo precisa capturar regularidades de diferentes tipos:

- ortografia, morfologia e sintaxe;

- associações semânticas e relações discursivas;
- estruturas recorrentes em código, fórmulas e documentos;
- padrões factuais presentes nos dados;
- regularidades sobre gêneros textuais e formas de interação.

Essas capacidades emergem como meios para reduzir a perda de predição. O objetivo não declara explicitamente "aprenda gramática" ou "memorize fatos". O modelo ajusta parâmetros que tornam as continuações observadas mais prováveis.

6.2 O modelo não é uma base de dados textual

O conhecimento adquirido fica distribuído nos parâmetros. O modelo não armazena, em geral, uma coleção de documentos recuperáveis literalmente por endereço. Essa compressão permite generalização e combinação, mas também produz limitações:

- fatos raros podem ser mal representados;
- informações conflitantes podem ser misturadas;
- o modelo pode produzir uma continuação plausível sem suporte factual;
- conhecimento posterior ao conjunto de treinamento não surge automaticamente;
- não existe garantia de que uma resposta probabilisticamente plausível seja verdadeira.

6.3 Escala, dados e capacidade

O termo "grande" em LLM refere-se principalmente ao número de parâmetros, à quantidade de dados e ao custo computacional. Aumentar esses recursos pode melhorar a capacidade de modelar padrões, mas não elimina problemas de qualidade dos dados, viés, avaliação ou especificação de objetivos.

Mais capacidade também aumenta a possibilidade de memorizar exemplos. Por isso, avaliação séria exige separar dados de treinamento e teste, controlar contaminação e examinar desempenho em distribuições diferentes daquelas usadas no ajuste.

Pergunta 6

Por que "prever o próximo token" pode produzir capacidades que parecem ir além de completar frases? Ao mesmo tempo, por que esse objetivo não garante compromisso com a verdade?

7 Ajuste Supervisionado para Seguir Instruções

Após o pré-treinamento, o modelo sabe atribuir probabilidades a continuações, mas pode não responder no formato esperado por um usuário. Um pedido pode ser continuado como parte de uma página, uma discussão ou uma lista de exemplos, em vez de ser tratado como uma instrução a executar.

O **ajuste supervisionado** (*Supervised Fine-Tuning*, SFT) utiliza exemplos da forma:

$$(\text{instrução},\ \text{resposta desejável}).$$

Os parâmetros do modelo pré-treinado servem como ponto inicial. O treinamento continua com uma taxa de aprendizado normalmente menor e com dados selecionados para demonstrar o comportamento esperado.

7.1 Função de perda do SFT

Se x representa a instrução e $y = (y_1, \ldots, y_T)$ representa a resposta demonstrada, a perda pode ser escrita como:

$$L_{\text{SFT}}(\theta) = -\sum_{t=1}^{T} \log P_\theta(y_t \mid x, y_{<t}).$$

Em muitas implementações, calcula-se a perda apenas sobre os tokens da resposta. Os tokens da instrução fornecem contexto, mas não são tratados como partes que o modelo deve aprender a reproduzir.

7.2 O que muda depois do SFT?

O modelo aprende padrões como:

- responder diretamente a perguntas;
- respeitar formatos solicitados;
- reconhecer papéis de sistema, usuário e assistente;
- recusar algumas solicitações;
- executar tarefas demonstradas nos exemplos.

Entretanto, o SFT ensina principalmente por imitação. Para cada instrução do conjunto, normalmente existe uma resposta demonstrada. Ele não informa explicitamente como comparar várias respostas plausíveis nem como equilibrar propriedades conflitantes, como concisão, completude, segurança e utilidade.

Além disso, a qualidade do comportamento fica limitada pela qualidade e pela diversidade das demonstrações. Se as respostas forem excessivamente longas, superficiais ou inconsistentes, o modelo poderá reproduzir esses padrões.

7.3 Exemplo: pré-treinamento e SFT

Considere a entrada:

> *Explique busca A* em três frases.*

Um modelo apenas pré-treinado pode continuar o texto como se estivesse reproduzindo um fórum, um livro ou uma lista de perguntas. Um modelo ajustado com exemplos de instruções tende a reconhecer que deve produzir uma explicação, controlar aproximadamente o tamanho e concluir a resposta.

O SFT não adiciona um mecanismo externo de planejamento nem garante que a explicação seja correta. Ele altera a distribuição de respostas para aproximá-la dos padrões presentes nas demonstrações.

Pergunta 7

Por que aumentar apenas a quantidade de dados de pré-treinamento pode não resolver a dificuldade de seguir uma instrução específica? Que informação adicional os pares instrução–resposta fornecem?

8 Aprendizado por Reforço com Feedback Humano

Os conceitos de política, recompensa e aproximação de funções também aparecem no treinamento de modelos de linguagem de grande porte (*Large Language Models*, ou LLMs). Um caso importante é o **Aprendizado por Reforço com Feedback Humano** (*Reinforcement Learning from Human Feedback*, ou RLHF).

Durante seu treinamento inicial, um modelo de linguagem aprende, essencialmente, a prever o próximo token a partir dos tokens anteriores. Esse objetivo permite que o modelo aprenda regularidades linguísticas e conhecimentos presentes nos dados, mas não garante que suas respostas sejam úteis, corretas, seguras ou compatíveis com as instruções do usuário. Produzir uma continuação provável não é necessariamente o mesmo que produzir uma resposta desejável.

O RLHF tenta reduzir essa diferença usando avaliações humanas como fonte de informação sobre o comportamento desejado (Christiano et al., 2017; Ouyang et al., 2022). Em vez de exigir que uma pessoa escreva uma função matemática capaz de atribuir recompensa a qualquer resposta possível, o método coleta comparações entre respostas produzidas pelo modelo. Essas comparações são usadas para aprender uma função de recompensa aproximada.

8.1 De preferências humanas a uma função de recompensa

Considere um prompt x e duas respostas candidatas y_A e y_B. Um avaliador humano pode indicar que prefere y_A a y_B:

$$y_A \succ y_B.$$

Essa comparação não fornece diretamente uma recompensa numérica. Ela fornece apenas uma relação de preferência. A partir de muitas comparações desse tipo, treina-se um *modelo de recompensa*:

$$\hat{r}_\phi(x, y),$$

onde x é o prompt, y é uma resposta e ϕ representa os parâmetros do modelo. O valor $\hat{r}_\phi(x, y)$ é uma estimativa escalar de quanto a resposta y parece satisfazer as preferências observadas durante o treinamento.

O modelo de recompensa é, portanto, uma função aproximada. Assim como a aproximação linear de $Q(s, a)$ estudada anteriormente, ele precisa generalizar a partir de exemplos limitados. Avaliadores humanos não conseguem examinar todas as respostas que um LLM pode produzir. O modelo de recompensa deve aprender regularidades nas comparações disponíveis e aplicá-las a respostas nunca avaliadas diretamente.

Essa generalização é necessária, mas introduz um risco. O modelo de recompensa não representa perfeitamente as preferências humanas. Ele representa apenas o que conseguiu aprender a partir dos exemplos fornecidos.

Uma forma comum de transformar as comparações em um objetivo de treinamento é o modelo de Bradley–Terry. Para uma resposta preferida y_w e uma rejeitada y_l, supõe-se que a probabilidade da preferência seja

$$P_\phi(y_w \succ y_l \mid x) = \sigma(\hat{r}_\phi(x, y_w) - \hat{r}_\phi(x, y_l)) \,,$$

o que produz a perda

$$L_{\mathrm{RM}}(\phi) = -\log \sigma(\hat{r}_\phi(x, y_w) - \hat{r}_\phi(x, y_l)) \,.$$

Assim, as comparações treinam diferenças entre recompensas, e não valores absolutos isolados. Adicionar a mesma constante a todas as recompensas não altera as probabilidades previstas; a origem da escala é, portanto, arbitrária. A escala relativa depende da parametrização e, quando introduzida, de uma temperatura ou parâmetro de ruído.

Pergunta 8

Por que não pedir que avaliadores humanos atribuam diretamente uma recompensa a cada resposta produzida durante o treinamento e durante o uso do LLM? Que papel o modelo de recompensa desempenha na escalabilidade do processo?

8.2 O processo de RLHF

Uma forma clássica de RLHF utiliza três etapas principais:

1. **Ajuste supervisionado.** O modelo inicial é treinado com exemplos de prompts acompanhados de respostas desejáveis. Essa etapa produz uma política inicial, frequentemente denominada π_{ref} ou política de referência.

2. **Aprendizado do modelo de recompensa.** Para um mesmo prompt, o modelo produz diferentes respostas. Avaliadores humanos ordenam ou comparam essas respostas. As preferências coletadas são usadas para ajustar $\hat{r}_\phi(x, y)$.

3. **Otimização da política.** O modelo de linguagem é tratado como uma política π_θ. Seus parâmetros são ajustados para aumentar a recompensa prevista por $\hat{r}_\phi$.

Somente a terceira etapa corresponde propriamente ao aprendizado por reforço. A primeira usa aprendizado supervisionado. A segunda também constitui um problema supervisionado, pois o modelo de recompensa aprende a partir das comparações fornecidas pelos avaliadores.

Se a política fosse ajustada apenas para maximizar a recompensa prevista, ela poderia afastar-se excessivamente do modelo de referência e explorar falhas do modelo de recompensa. Por isso, a otimização costuma considerar dois objetivos:

aumentar a recompensa prevista e evitar mudanças excessivas na política.

Uma formulação conceitual desse compromisso é:

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathcal{D}} \left[\mathbb{E}_{y\sim\pi_\theta(\cdot|x)} \left[\hat{r}_\phi(x, y) \right] - \beta \, D_{\mathrm{KL}} \left(\pi_\theta(\cdot \mid x) \parallel \pi_{\mathrm{ref}}(\cdot \mid x) \right) \right].$$

O primeiro termo favorece respostas com recompensa prevista elevada. O segundo penaliza o afastamento em relação à política de referência. A divergência D_{KL} é assimétrica: a ordem das duas políticas importa. Na prática, a expectativa e a divergência são estimadas por amostras de prompts e respostas; em políticas autoregressivas, a razão de probabilidades de uma resposta se decompõe em termos por token. Os logaritmos são naturais, de modo que a perda e a divergência são medidas em nats. O coeficiente β controla a intensidade da penalidade.

Nas seções seguintes, abriremos essa caixa-preta em nível conceitual e veremos como o PPO usa uma razão de probabilidades e um objetivo com *clipping* para realizar atualizações mais conservadoras. O ponto central é que a política do LLM é ajustada para aumentar uma recompensa aprendida, mantendo alguma proximidade com o comportamento adquirido durante o ajuste supervisionado.

8.3 RLHF como um problema de aprendizado por reforço

A geração de texto pode ser representada como um processo sequencial. A cada passo, o modelo observa o prompt e os tokens já produzidos, escolhe o próximo token e acrescenta esse token à resposta.

A Tabela 1 relaciona os elementos desse processo aos conceitos de aprendizado por reforço.

Nessa formulação, a função de transição é simples: o token escolhido é acrescentado ao texto já existente. A principal incerteza está na recompensa. Não existe uma função conhecida que determine exatamente quão útil, correta ou segura é qualquer resposta possível. O RLHF tenta aprender uma aproximação dessa função a partir de preferências humanas.

Também é possível adotar uma abstração mais simples e tratar a resposta completa como uma única ação. Nesse caso, o prompt é o contexto, cada resposta possível é uma ação e a recompensa é atribuída ao final. Essa formulação se aproxima de um *bandit* contextual.

Conceito de AR	**Interpretação na geração de texto**
Estado s_t	Prompt e sequência de tokens já produzida até o instante t
Ação a_t	Próximo token escolhido pelo modelo
Política $\pi_\theta(a_t \mid s_t)$	Distribuição de probabilidades sobre os possíveis próximos tokens
Transição	A ação escolhida é acrescentada à sequência, produzindo o próximo estado
Episódio	Geração completa de uma resposta
Recompensa	Avaliação humana ou pontuação fornecida pelo modelo de recompensa

Tabela 1: Correspondência entre os componentes de um problema de aprendizado por reforço e a geração de texto por um LLM.

A formulação token a token é mais fiel ao processo de geração, mas torna explícito o problema de atribuição de crédito: se uma resposta recebe uma avaliação ruim, quais tokens ou decisões tomadas durante sua geração foram responsáveis pelo resultado?

É importante observar que o RLHF não utiliza necessariamente Q-learning ou SARSA. Esses algoritmos aprendem valores de estado-ação a partir de alvos de Diferença Temporal. No RLHF clássico, costuma-se ajustar diretamente a política do modelo de linguagem. Apesar dessa diferença algorítmica, permanecem os mesmos elementos fundamentais: política, interação, recompensa, aproximação de funções e otimização do comportamento esperado.

8.4 Exemplo resolvido: aprendizado de preferências sobre respostas

Considere o seguinte prompt:

> *Explique o que é overfitting em no máximo duas frases.*

Um modelo de linguagem produz três respostas:

- $\boldsymbol{y_1}$: *Overfitting ocorre quando um modelo se ajusta excessivamente aos dados de treinamento, incluindo seus ruídos e particularidades. Com isso, ele apresenta bom desempenho no treinamento, mas generaliza mal para novos dados.*
- $\boldsymbol{y_2}$: *Overfitting ocorre quando um modelo aprende muito bem os dados de treinamento. Ele pode inclusive memorizar particularidades desses dados. Seu desempenho no treinamento costuma ser elevado. Entretanto, o desempenho em novos dados pode ser ruim. Técnicas de regularização podem ajudar a reduzir o problema.*

y_3: *Overfitting ocorre quando o modelo é simples demais para aprender os padrões existentes nos dados. Por isso, ele apresenta desempenho ruim tanto no treinamento quanto no teste.*

A primeira resposta é correta, clara e respeita o limite de duas frases. A segunda também é conceitualmente correta, mas não segue a restrição de tamanho. A terceira respeita o tamanho, porém descreve *underfitting*, não *overfitting*. Considere que, com base nessa análise, um avaliador humano estabelece a seguinte preferência:

$$y_1 \succ y_2 \succ y_3.$$

Para tornar o exemplo transparente, suponha um modelo de recompensa linear com três características:

$$\hat{r}(x, y) = 2f_{\text{cor}}(x, y) + f_{\text{inst}}(x, y) + 0{,}5f_{\text{clar}}(x, y),$$

onde:

- f_{cor} representa a correção conceitual estimada;
- f_{inst} representa o cumprimento da instrução;
- f_{clar} representa a clareza da resposta.

Na prática, modelos de recompensa não precisam utilizar características explicitamente definidas por seres humanos. Eles podem aprender representações internas a partir dos próprios textos. O modelo linear é usado aqui apenas para tornar o cálculo visível.

Suponha que o modelo atribua os seguintes valores:

Resposta	f_{cor}	f_{inst}	f_{clar}	$\hat{r}(x, y)$
y_1	1	1	0,8	3,40
y_2	1	0	0,9	2,45
y_3	0	1	0,7	1,35

Para y_1, temos:

$$\hat{r}(x, y_1) = 2(1) + 1 + 0{,}5(0{,}8) = 3{,}40.$$

Para y_2:

$$\hat{r}(x, y_2) = 2(1) + 0 + 0{,}5(0{,}9) = 2{,}45.$$

Para y_3:

$$\hat{r}(x, y_3) = 2(0) + 1 + 0{,}5(0{,}7) = 1{,}35.$$

Portanto:

$$\hat{r}(x, y_1) > \hat{r}(x, y_2) > \hat{r}(x, y_3).$$

Os valores atribuídos pelo modelo de recompensa são consistentes com a ordem estabelecida pelos avaliadores. Durante a etapa de aprendizado por reforço, respostas semelhantes a y_1 receberão recompensa prevista maior. A otimização tenderá, portanto, a aumentar sua probabilidade. Respostas semelhantes a y_2 ou y_3 tenderão a perder probabilidade.

Observe que o agente não recebeu uma resposta correta para copiar durante essa etapa. Ele recebeu um sinal escalar que indica quais respostas parecem preferíveis. Esse sinal orienta a modificação da política.

Quando a recompensa aproximada falha

Suponha agora que o modelo produza uma quarta resposta:

> $\boldsymbol{y_4}$: *Overfitting ocorre sempre que a acurácia de treinamento é maior que a acurácia de teste. O problema pode ser completamente eliminado aumentando o conjunto de treinamento.*

A resposta é curta, clara e usa termos associados ao tema. Entretanto, suas duas afirmações são excessivas: uma diferença entre treinamento e teste não caracteriza, por si só, qualquer caso de *overfitting*, e aumentar o conjunto de treinamento não garante a eliminação completa do problema.

Suponha que, apesar disso, o modelo de recompensa interprete a linguagem confiante e o vocabulário técnico como evidências de correção:

$$f_{\text{cor}}(x, y_4) = 1, \qquad f_{\text{inst}}(x, y_4) = 1, \qquad f_{\text{clar}}(x, y_4) = 1.$$

Nesse caso:

$$\hat{r}(x, y_4) = 2(1) + 1 + 0{,}5(1) = 3{,}50.$$

Logo:

$$\hat{r}(x, y_4) > \hat{r}(x, y_1),$$

embora avaliadores humanos bem informados prefiram y_1.

Se a política explorar sistematicamente essa falha, poderá aprender a produzir respostas confiantes e tecnicamente plausíveis, mas incorretas. O algoritmo de otimização não cometeu um erro: ele aumentou a recompensa que recebeu. O problema está no fato de que a recompensa aprendida é apenas uma aproximação do objetivo humano.

Esse fenômeno é denominado *exploração da recompensa* ou *reward hacking*. O agente encontra uma forma de aumentar a recompensa prevista sem realizar adequadamente aquilo que o projetista pretendia incentivar.

Pergunta 9

No exemplo anterior, o modelo de linguagem aprendeu a produzir respostas como y_4, que recebem recompensa elevada apesar de conterem afirmações incorretas. Analise o problema respondendo: **(a)** por que a política está agindo racionalmente em relação à recompensa recebida; **(b)** que erro de generalização foi cometido pelo modelo de recompensa; **(c)** que novos exemplos de preferência poderiam ajudar a corrigir esse erro; e **(d)** por que aumentar a capacidade do modelo de linguagem, sem modificar o modelo de recompensa, poderia agravar o problema.

8.5 Limitações e cuidados

O RLHF permite transmitir objetivos difíceis de expressar por meio de uma função de recompensa escrita manualmente. Em muitos casos, é mais fácil para uma pessoa comparar duas respostas do que atribuir uma pontuação absoluta e precisa a cada uma delas. Ainda assim, o método possui limitações importantes.

Primeiro, as preferências coletadas dependem dos avaliadores, das instruções que eles recebem e dos exemplos apresentados. Pessoas diferentes podem discordar sobre clareza, utilidade, segurança ou adequação. O modelo de recompensa aprende uma aproximação das preferências presentes nos dados, não uma definição universal de comportamento correto.

Segundo, o modelo de recompensa precisa avaliar respostas diferentes das que apareceram em seu treinamento. Como qualquer aproximador, ele pode generalizar de forma indevida. Quanto mais capaz for a política de buscar respostas com recompensa elevada, maior pode ser sua capacidade de encontrar regiões em que a aproximação falha.

Terceiro, a recompensa costuma ser atribuída à resposta completa. Isso torna difícil identificar quais decisões tomadas durante a geração foram responsáveis pela avaliação final. Trata-se novamente do problema de atribuição de crédito.

Por fim, uma recompensa prevista elevada não garante que a resposta seja verdadeira, segura ou útil. Ela garante apenas que o modelo de recompensa reconheceu naquela resposta características associadas a avaliações positivas. O RLHF não elimina o problema de especificar objetivos. Ele transfere parte desse problema para a coleta de preferências e para o aprendizado da função de recompensa.

8.6 RLHF e *reward shaping*

Embora ambos utilizem recompensas para orientar o comportamento, RLHF e *reward shaping* não são a mesma técnica.

No *reward shaping*, parte-se de uma recompensa já definida e adiciona-se um termo intermediário para acelerar o aprendizado. Esse termo pode ser projetado manualmente ou obtido de outra fonte. A afirmação de que a política ótima não muda exige uma condição específica: no *potential-based shaping*, o termo tem a forma $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$, sob as hipóteses usuais do MDP, e então preserva as políticas ótimas (Ng et al., 1999). Um termo intermediário arbitrário pode alterar a política ótima. No RLHF, o problema central é diferente: a recompensa correspondente ao comportamento desejado não está

disponível em forma matemática. Ela precisa ser aproximada a partir de comparações humanas.

Nos dois casos, porém, permanece a mesma advertência: o agente aprende a maximizar o sinal que recebe, não necessariamente a intenção que motivou a construção desse sinal. Uma recompensa mal projetada ou mal aproximada pode produzir comportamentos indesejados mesmo quando o algoritmo de aprendizado funciona corretamente.

Também existem métodos de alinhamento por preferências que não executam explicitamente a etapa de aprendizado por reforço. Esses métodos ajustam a política diretamente a partir de pares de respostas preferidas e rejeitadas. Portanto, nem todo treinamento com preferências humanas deve ser chamado, em sentido estrito, de RLHF. O RLHF é uma abordagem específica dentro de uma família mais ampla de métodos de alinhamento de modelos de linguagem.

Pergunta 10

Compare RLHF e *reward shaping*. O *reward shaping* exige necessariamente informação humana? Em que condição o termo de shaping preserva a política ótima, e quando pode alterá-la? Qual das duas técnicas procura aprender uma aproximação de um objetivo que não foi fornecido explicitamente?

9 Otimização da Política com PPO

Na seção anterior, o algoritmo de otimização da política foi mantido como uma caixa-preta. Agora abriremos essa caixa em nível conceitual. Um método clássico usado em RLHF é o **Proximal Policy Optimization** (PPO), introduzido por Schulman et al. (2017). Seu objetivo é melhorar a política sem permitir que uma única atualização a modifique de forma excessiva em relação à política que gerou os dados.

Essa preocupação é particularmente importante em LLMs. O modelo que inicia o RLHF já passou por pré-treinamento e ajuste supervisionado. O *clipping* controla a estabilidade da atualização em relação à política antiga e à distribuição que gerou as amostras. A penalização por KL em relação a π_{ref}, discutida adiante, cumpre outro papel: desencoraja o afastamento acumulado de capacidades e comportamentos adquiridos anteriormente.

9.1 Otimização direta de uma política

Em Q-learning, a política é derivada dos valores de ação. Em métodos de gradiente de política, os próprios parâmetros da política são ajustados para aumentar o retorno esperado:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right],$$

onde τ representa uma trajetória gerada pela política. No contexto de um LLM, a trajetória pode ser a sequência completa de tokens de uma resposta.

Uma forma conceitual do gradiente de política é:

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A_t\right].$$

O termo A_t, denominado **vantagem**, indica se a ação tomada no estado s_t produziu resultado melhor ou pior do que uma referência esperada:

- se $A_t > 0$, aumenta-se a probabilidade da ação observada;
- se $A_t < 0$, diminui-se sua probabilidade;
- quanto maior $|A_t|$, maior tende a ser a correção.

Em aplicações de PPO, uma função de valor estimada por um **crítico** ajuda a construir essa referência. O crítico não escolhe diretamente os tokens. Ele estima o retorno esperado e ajuda a reduzir a variabilidade da atualização da política.

9.2 Política antiga e razão de probabilidades

Antes de um conjunto de atualizações, mantém-se uma cópia da política que gerou as respostas, denominada $\pi_{\theta_{\text{old}}}$. Para cada ação observada, calcula-se a razão:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

Essa razão mede quanto a nova política alterou a probabilidade da ação:

- $r_t = 1$: a probabilidade não mudou;
- $r_t > 1$: a ação ficou mais provável;
- $r_t < 1$: a ação ficou menos provável.

Se uma ação possui vantagem positiva, o treinamento deseja aumentar sua probabilidade. Entretanto, permitir que r_t cresça sem controle pode produzir uma atualização instável.

9.3 O objetivo com *clipping*

O PPO limita o benefício obtido por uma alteração muito grande na razão de probabilidades. Uma forma de seu objetivo é:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)A_t, \text{clip}\big(r_t(\theta), 1-\varepsilon, 1+\varepsilon\big)A_t\right)\right].$$

A função clip restringe a razão ao intervalo

$$[1-\varepsilon,\, 1+\varepsilon].$$

O objetivo não proíbe matematicamente que a política se mova além do intervalo. Ele remove o incentivo fornecido pelo objetivo substituto para mudanças que ultrapassem a região considerada segura. Por isso, o nome *proximal*: procura-se uma política melhor, mas ainda próxima da política que gerou os dados.

9.4 Exemplo resolvido: efeito do *clipping*

Suponha que uma ação tenha:

$$\pi_{\theta_{\text{old}}}(a_t \mid s_t) = 0{,}20, \qquad \pi_\theta(a_t \mid s_t) = 0{,}30,$$

com vantagem $A_t = 2$ e $\varepsilon = 0{,}20$. A razão será:

$$r_t(\theta) = \frac{0{,}30}{0{,}20} = 1{,}50.$$

Sem *clipping*, a contribuição seria:

$$r_t A_t = 1{,}50 \times 2 = 3{,}00.$$

O limite superior permitido é:

$$1 + \varepsilon = 1{,}20.$$

Assim, a contribuição limitada é:

$$\text{clip}(1{,}50, 0{,}80, 1{,}20) \times 2 = 1{,}20 \times 2 = 2{,}40.$$

O objetivo utiliza o menor valor:

$$\min(3{,}00, 2{,}40) = 2{,}40.$$

A ação já recebeu o incentivo máximo previsto para essa amostra: como o termo clipado é constante em relação a θ nessa região, o excedente não produz benefício adicional nem gradiente para continuar aumentando sua probabilidade. O *clipping* limita o incentivo a uma mudança abrupta; ele não é a penalização KL que ancora a política na referência.

Pergunta 11

Considere $\pi_{\theta_{\text{old}}}(a_t \mid s_t) = 0{,}50$, $\pi_\theta(a_t \mid s_t) = 0{,}35$, $A_t = -3$ e $\varepsilon = 0{,}20$. Calcule r_t, os dois termos do objetivo PPO e o mínimo entre eles. Explique por que o caso de vantagem negativa exige cuidado ao interpretar o *clipping*.

9.5 Duas noções diferentes de proximidade

No RLHF com PPO aparecem duas políticas de comparação que não devem ser confundidas:

1. $\pi_{\theta_{\text{old}}}$ é a política que gerou as amostras usadas na atualização atual. A razão de probabilidades e o *clipping* do PPO comparam a política nova com essa cópia recente.

2. π_{ref} é, em geral, uma política de referência fixa, frequentemente derivada do SFT. A penalização por divergência KL impede que o modelo otimizado se afaste excessivamente desse comportamento de referência.

A primeira comparação busca estabilidade local entre atualizações sucessivas. A segunda preserva uma âncora comportamental ao longo de todo o processo de RLHF.

Uma recompensa regularizada pode ser escrita conceitualmente como:

$$R(x, y) = \hat{r}_\phi(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

O primeiro termo vem do modelo de recompensa. O segundo penaliza respostas às quais a política atual atribui probabilidade muito maior que a política de referência. O coeficiente β controla o compromisso entre explorar a recompensa aprendida e preservar o comportamento anterior.

9.6 Fluxo conceitual do RLHF com PPO

Uma iteração simplificada pode ser descrita assim:

1. selecionar um conjunto de prompts;
2. gerar respostas com a política atual;
3. calcular a pontuação do modelo de recompensa;
4. aplicar a penalização de divergência em relação à política de referência;
5. estimar retornos e vantagens;
6. atualizar a política com o objetivo limitado do PPO;
7. atualizar o crítico para melhorar suas estimativas de valor;
8. repetir o processo com novas respostas.

Esse procedimento é mais complexo que o ajuste supervisionado. As respostas precisam ser geradas durante o treinamento, o modelo de recompensa precisa avaliá-las e diferentes componentes devem ser mantidos em memória. Além disso, os resultados dependem de hiperparâmetros como taxa de aprendizado, ε e β.

PPO não resolve as limitações do modelo de recompensa. Ele apenas fornece uma forma relativamente conservadora de otimizar a política em relação ao sinal disponível. Se esse sinal estiver errado, uma otimização estável ainda poderá produzir o comportamento errado.

10 Otimização Direta de Preferências

O RLHF clássico decompõe o processo em duas partes: aprende um modelo de recompensa e depois utiliza aprendizado por reforço para otimizar a política. Métodos de **Otimização Direta de Preferências** procuram ajustar a política diretamente a partir dos pares comparados.

Um exemplo é o *Direct Preference Optimization* (DPO), proposto por Rafailov et al. (2023). Para cada prompt x, o conjunto de treinamento contém uma resposta preferida y_w e uma resposta rejeitada y_l:

$$(x, y_w, y_l).$$

O DPO aumenta, em relação à política de referência, a preferência da política atual por y_w sobre y_l. Uma forma de sua perda é:

$$L_{\mathrm{DPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right]\right)\right],$$

onde σ é a função sigmoide. A expressão compara quanto a política atual elevou a resposta preferida e a resposta rejeitada em relação à mesma referência.

O DPO ainda precisa de uma política de referência π_{ref}, normalmente obtida a partir do modelo ajustado por SFT. Na forma básica, os pares (x, y_w, y_l) são mantidos em um conjunto fixo e a política é ajustada de modo *offline*; não há coleta contínua de novas respostas da política atual durante cada atualização. Isso torna possível um deslocamento de distribuição entre os pares disponíveis e as respostas que a política otimizada passa a produzir. Além disso, a interpretação do DPO como uma forma direta de otimizar o objetivo regularizado do RLHF depende da hipótese de preferências do tipo Bradley–Terry e de outras condições do modelo, não sendo uma equivalência universal para quaisquer dados de preferência.

10.1 O que o DPO remove?

Na forma básica, o DPO não exige:

- treinar e servir separadamente um modelo explícito de recompensa durante a otimização;
- gerar novas trajetórias a cada atualização da política;
- estimar vantagens com um crítico;
- executar o laço de PPO.

Isso torna o treinamento mais próximo de uma tarefa supervisionada sobre pares de preferência. Contudo, o sinal continua dependendo da qualidade das comparações humanas ou sintéticas. O DPO simplifica a otimização; não elimina problemas de viés, cobertura, ambiguidade ou generalização das preferências.

10.2 RLHF e DPO não são sinônimos

No uso informal, a expressão RLHF às vezes designa qualquer pós-treinamento com dados de preferência. Em sentido estrito, porém, DPO não executa um algoritmo explícito de aprendizado por reforço. É mais preciso utilizar **alinhamento por preferências** como categoria ampla e distinguir dentro dela:

- RLHF com modelo de recompensa e otimização por reforço;
- otimização direta de preferências;
- variantes que usam feedback produzido por outros modelos, regras ou verificadores.

Pergunta 12

O DPO remove o modelo explícito de recompensa e o laço de PPO. Isso significa que ele remove a necessidade de representar preferências ou elimina o risco de aprender preferências inadequadas? Justifique.

11 Comparação das Etapas de Treinamento

A Tabela 2 sintetiza os objetivos e os sinais usados nas principais etapas discutidas.

Etapa	Dados	Sinal de treinamento	Resultado buscado
Pré-treinamento	Texto não rotulado manualmente	Próximo token observado	Modelar regularidades gerais da linguagem e dos dados
SFT	Pares instrução–resposta	Tokens da resposta demonstrada	Seguir instruções e imitar respostas desejáveis
Modelo de recompensa	Comparações entre respostas	Ordem de preferência	Aproximar preferências por uma pontuação escalar
RLHF com PPO	Prompts, respostas geradas e recompensa aprendida	Vantagem e recompensa regularizada	Aumentar a recompensa prevista sem mudar abruptamente a política
DPO	Resposta preferida e resposta rejeitada	Contraste entre o par	Ajustar diretamente a política às preferências

Tabela 2: Comparação entre as etapas de treinamento e pós-treinamento de um modelo de linguagem.

As etapas não são intercambiáveis. Pré-treinamento não substitui demonstrações de comportamento. SFT não expressa todas as comparações possíveis entre respostas. RLHF e

DPO não fornecem conhecimento factual perfeito. Cada etapa modifica a política segundo um sinal específico e herda as limitações desse sinal.

11.1 Uma visão integrada

A Figura 2 mostra uma visão simplificada do processo completo.

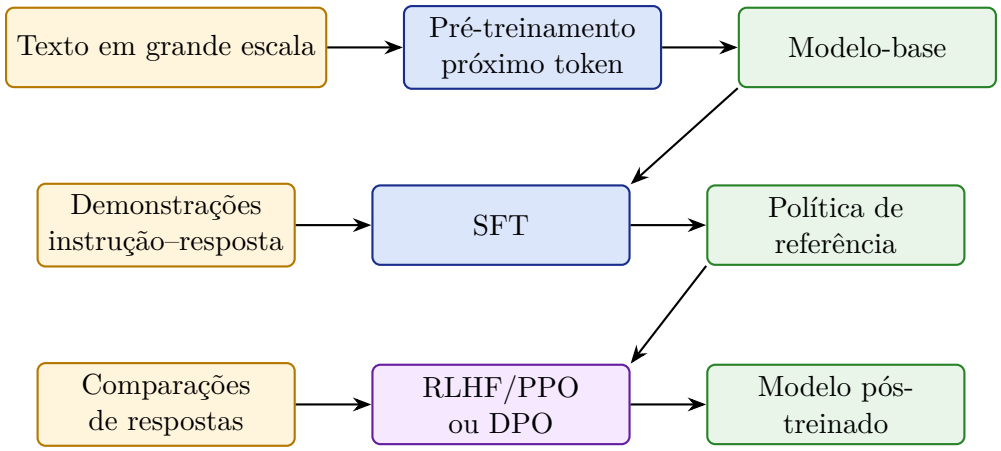


Figura 2: Visão simplificada do treinamento de um modelo de linguagem. Cada etapa parte do modelo produzido anteriormente e utiliza um tipo diferente de sinal.

O diagrama não implica que exista uma única receita industrial. Conjuntos de dados, objetivos e algoritmos variam. A separação é conceitual: ela ajuda a identificar que informação entra em cada etapa e qual comportamento pode ser esperado como consequência.

12 Limitações e Fronteiras do Alinhamento

Pós-treinamento não transforma automaticamente um modelo probabilístico em uma fonte confiável de verdade. Diversas limitações permanecem:

- **Especificação incompleta**: critérios como utilidade e segurança admitem interpretações diferentes.
- **Desacordo entre avaliadores**: preferências variam entre pessoas, culturas e contextos.
- **Generalização do feedback**: avaliações cobrem apenas uma pequena fração das entradas possíveis.
- **Exploração da recompensa**: políticas capazes podem encontrar falhas sistemáticas do modelo de recompensa.
- **Deslocamento de distribuição**: o modelo pode ser usado em tarefas diferentes das observadas no pós-treinamento.
- **Avaliação circular**: modelos podem ser usados para avaliar outros modelos, propagando vieses e erros.

- **Conflito entre objetivos**: concisão, completude, segurança e criatividade nem sempre podem ser maximizadas simultaneamente.

Essas limitações não tornam o alinhamento inútil. Elas mostram que o problema não se reduz a selecionar um algoritmo. Dados, critérios, avaliação externa e monitoramento continuam necessários.

Pergunta 13

Suponha que avaliadores prefiram sistematicamente respostas longas porque elas parecem mais completas. Que comportamento pode emergir após o pós-treinamento? Como distinguir uma resposta realmente mais útil de uma resposta que apenas explora essa regularidade das preferências?

13 Do Modelo de Linguagem ao Agente

Um modelo de linguagem implementa, essencialmente, uma política de continuação condicionada ao contexto. Ele não possui, por si só, acesso garantido a ferramentas, memória persistente, fontes atualizadas ou um ambiente externo.

Um **agente baseado em LLM** acrescenta uma arquitetura de controle ao redor do modelo. Essa arquitetura pode:

- manter um estado da tarefa;
- solicitar planos e revisar decisões;
- escolher e executar ferramentas;
- observar os resultados das ações;
- recuperar documentos externos;
- registrar memória;
- aplicar políticas de segurança e critérios de parada.

A passagem de LLM para agente não exige necessariamente novo treinamento dos parâmetros. Muitas capacidades agentivas surgem da organização do ciclo de inferência e da integração com componentes externos. O pós-treinamento influencia quão bem o modelo segue instruções, produz chamadas estruturadas e responde ao feedback, mas não substitui a engenharia do sistema agentivo.

Essa distinção prepara a próxima parte da disciplina, dedicada a ReAct, planejamento iterativo, ferramentas, RAG, MCP, observabilidade, avaliação e *guardrails*.

No cenário desta seção, suponha que o LLM seja responsável por propor quando uma ferramenta deve ser chamada e com quais argumentos, como ocorre em sistemas com *function calling*. A arquitetura do agente continua responsável por disponibilizar as ferramentas, validar a chamada, executá-la, controlar permissões e reinserir o resultado no

contexto. Outras arquiteturas podem usar um roteador externo, mas essa hipótese precisa ser explicitada.

Pergunta 14

Um sistema recebe uma pergunta, consulta automaticamente uma base documental, executa código e revisa a resposta antes de entregá-la. Assumindo que o LLM propõe as chamadas de ferramenta, quais capacidades pertencem ao LLM e quais pertencem à arquitetura do agente?

14 Resumo

Modelos de linguagem representam textos como sequências de tokens convertidos em vetores. Blocos Transformer utilizam autoatenção causal para combinar informações do contexto e produzir logits sobre o próximo token. O pré-treinamento ajusta os parâmetros com um objetivo autossupervisionado de predição, permitindo que o modelo aprenda regularidades gerais dos dados.

O ajuste supervisionado modifica o modelo-base para imitar respostas demonstradas e seguir instruções. Dados de preferência acrescentam um novo tipo de informação: comparações entre respostas plausíveis. No RLHF clássico, essas comparações treinam um modelo de recompensa, e uma política é otimizada em relação à recompensa aprendida. O PPO limita mudanças excessivas por meio de uma razão de probabilidades e de um objetivo com *clipping*; uma penalização KL mantém a política próxima de uma referência. O DPO utiliza diretamente pares de preferência e evita o laço explícito de recompensa mais PPO.

Nenhuma dessas etapas garante verdade, segurança ou alinhamento universal. Cada uma otimiza um sinal limitado. O modelo pós-treinado pode, então, ser usado como componente de um agente, mas o ciclo agentivo, as ferramentas, a memória e os mecanismos de controle pertencem à arquitetura do sistema.

Pergunta 15

Explique por que a sequência "pré-treinamento → SFT → alinhamento por preferências" não deve ser interpretada como três formas equivalentes de ensinar o mesmo conteúdo ao modelo. Identifique o sinal, o objetivo e uma limitação de cada etapa.

15 Atividades Intra-Classe

As atividades devem ser resolvidas em grupo. O foco está em explicar as relações entre representação, objetivo de treinamento e comportamento produzido.

15.1 Aquecimento

(a) Qual é a diferença entre um token e um embedding?

(b) O que os logits representam antes da aplicação do softmax?

(c) Por que o pré-treinamento autoregressivo é denominado autossupervisionado?

(d) Qual informação existe em dados de preferência que não aparece em um único par instrução–resposta?

(e) Qual é a diferença entre $\pi_{\theta_{\text{old}}}$ e π_{ref} no RLHF com PPO?

(f) DPO é, em sentido estrito, um algoritmo de aprendizado por reforço? Justifique.

15.2 Probabilidade de sequências

Considere um modelo com o seguinte comportamento:

- $P(\texttt{IA} \mid \texttt{A}) = 0{,}6$;
- $P(\texttt{aprende} \mid \texttt{A IA}) = 0{,}5$;
- $P(\texttt{age} \mid \texttt{A IA}) = 0{,}3$;
- $P(\texttt{com dados} \mid \texttt{A IA aprende}) = 0{,}4$;
- $P(\texttt{no ambiente} \mid \texttt{A IA age}) = 0{,}7$.

(a) Calcule a probabilidade da sequência “A IA aprende com dados” segundo os valores fornecidos.

(b) Calcule a probabilidade da sequência “A IA age no ambiente”.

(c) Qual sequência é globalmente mais provável entre as duas?

(d) O que o exemplo revela sobre a diferença entre comparar apenas o próximo token e comparar sequências completas?

15.3 Atenção e máscara causal

Considere a sequência de cinco tokens:

$$x_1, x_2, x_3, x_4, x_5.$$

(a) Durante o cálculo da representação de x_3 em um Transformer causal, quais posições podem receber atenção?

(b) Por que permitir atenção a x_4 e x_5 invalidaria o objetivo de predição autoregressiva?

(c) Se a atenção de x_5 atribui pesos 0,1, 0,2, 0,1, 0,3 e 0,3 às cinco posições, o que esses valores permitem afirmar? O que eles não permitem concluir sobre a causa da previsão?

15.4 Pré-treinamento, SFT e preferências

Para cada situação, identifique a etapa mais diretamente relacionada e justifique:

(a) O modelo completa corretamente uma expressão idiomática rara.

(b) O modelo passa a responder em JSON quando solicitado.

(c) O modelo aprende que, entre duas respostas corretas, avaliadores preferem a mais direta.

(d) O modelo produz respostas plausíveis, porém inventa referências. Discuta se esse comportamento pode estar relacionado ao objetivo de pré-treinamento, à falta de demonstrações adequadas no SFT e à ausência de verificadores ou preferências que penalizem a invenção.

(e) O modelo recusa uma classe de solicitações após receber demonstrações de recusa.

15.5 PPO

Considere uma ação com vantagem $A_t = 1{,}5$, probabilidade antiga 0,25, probabilidade nova 0,35 e $\varepsilon = 0{,}20$.

(a) Calcule $r_t(\theta)$.

(b) Calcule $r_t A_t$.

(c) Calcule o termo com *clipping*.

(d) Qual termo será usado pelo objetivo do PPO?

(e) Interprete o resultado: a ação recebe incentivo até o limite? Quando o termo clipado é o menor e fica constante em relação a θ, por que essa amostra deixa de fornecer gradiente adicional para aumentar ainda mais sua probabilidade?

15.6 Projeto de dados de preferência

Considere um assistente acadêmico que deve responder dúvidas sobre uma disciplina. Proponha quatro pares de respostas para treinar preferências, cobrindo conflitos diferentes:

(a) correção versus linguagem confiante;

(b) concisão versus completude;

(c) resposta direta versus pedido legítimo de esclarecimento;

(d) utilidade versus risco de fornecer uma solução integral de atividade avaliativa.

Para cada par, indique qual resposta deve ser preferida e que regularidade o modelo deveria aprender.

15.7 Perguntas para discussão

Pergunta 16

Se um modelo de recompensa alcança alta acurácia na comparação de respostas de validação, isso garante que será seguro usá-lo para otimizar uma política muito mais capaz? Discuta o efeito da busca ativa da política por regiões em que o modelo de recompensa erra.

Pergunta 17

Temperatura, SFT e PPO podem modificar o comportamento observado de um LLM, mas atuam em momentos diferentes. Compare os três mecanismos e indique quais alteram parâmetros e qual atua apenas durante a geração.

Pergunta 18

Um sistema usa um LLM pós-treinado, uma base vetorial, um interpretador Python e um controlador que decide quando chamar cada ferramenta. Quais partes foram aprendidas no treinamento do LLM e quais pertencem ao projeto do agente?

Referências

Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg e Dario Amodei. Deep reinforcement learning from human preferences. Em *Advances in Neural Information Processing Systems*, 2017.

Andrew Y. Ng, Daishi Harada e Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. Em *Proceedings of the Sixteenth International Conference on Machine Learning*, 1999.

Long Ouyang et al. Training language models to follow instructions with human feedback. Em *Advances in Neural Information Processing Systems*, 2022.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning e Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. Em *Advances in Neural Information Processing Systems*, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford e Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

Ashish Vaswani et al. Attention is all you need. Em *Advances in Neural Information Processing Systems*, 2017.